

Graph -

$G(V, E)$

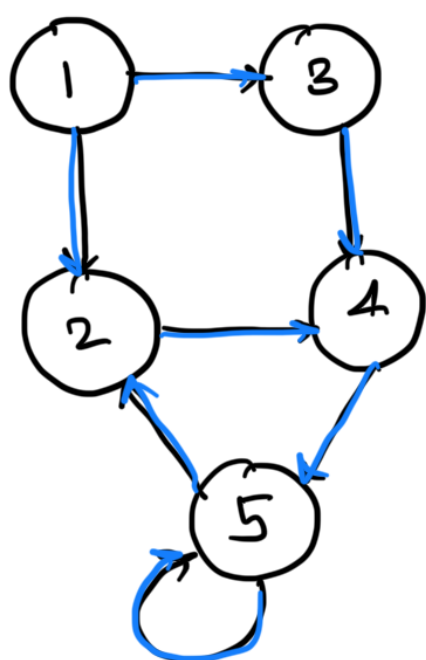
$V = \text{Vertex}$

$E = \text{Edge}$

- A graph is a non-linear data structure composed 2 components:

1. Vertex: (Node) : set of vertices

2. Edge: set of edges connecting pairs of vertices



$V = \{1, 2, 3, 4, 5\}$

$E = \{(1, 2), (1, 3), (3, 4), (2, 4), (5, 2), (4, 5), (5, 5)\}$

Directed Graph

Edge = Source $\longrightarrow$ Destⁿ.



Components of Graph Data Structure

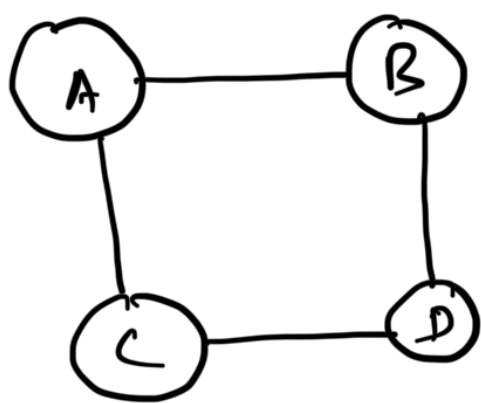


① Vertex

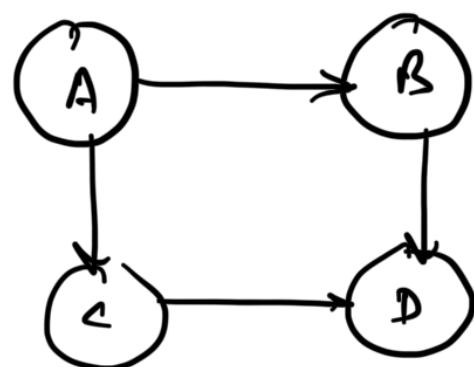
- fundamental unit of the graph
- Can be labeled or unlabeled
- Ex: Transportation network

② Edges

- Order pairs in a directed graph & unordered pair in a undirected graph
- Can represent relationship
- Ex. facebook



Undirected graph



Directed graph

$$E = \{ (A,B), (B,A), (B,D), (D,B), (C,D), (D,C), (A,C), (C,A) \}$$

$$E = 8$$

Undirected have a symmetric relation

$$E = \{ (A,B), (B,D), (C,D), (A,C) \}$$

$$E = 4$$

Graph Properties:

① Number of Edges - Undirected Graph

- The no. of possible pairs in an n vertex graph is $n * (n-1)$ ✓

- Since edge (u,v) is same as (v,u) the number of edges in an undirected graph is $\frac{n * (n-1)}{2}$ ✓

symmetric
 $\frac{8}{2} = 4$

② Number of Edges - Directed Graph

- The no. of possible pairs in an n

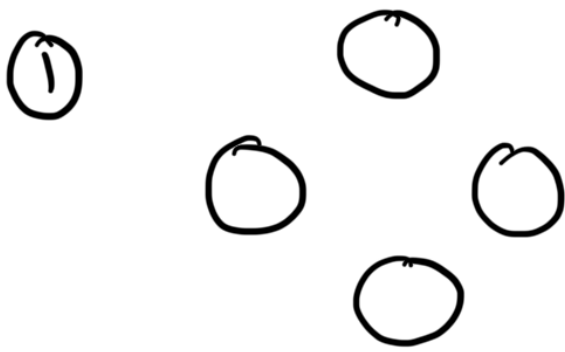
vertex graph is $n \times (n-1)$

- Edge (u,v) is not same as (v,u)

the number of edges in an directed graph is $\leq n \times (n-1)$ 4 edges

Types of Graph

$G(V,E)$



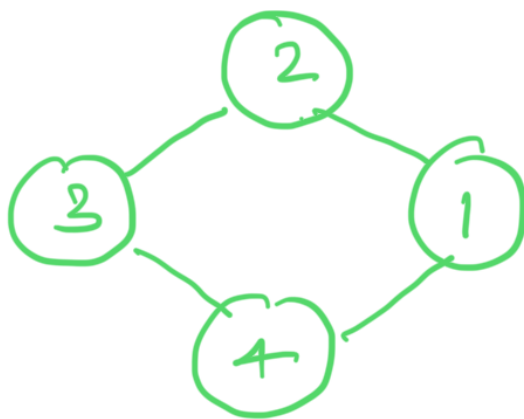
Null Graph →

No edges between the vertices



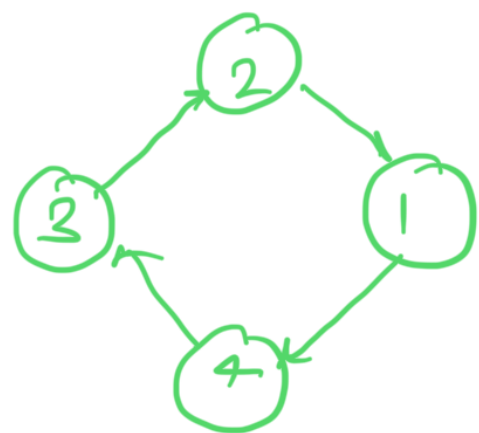
Trivial Graph

Graph with one vertex



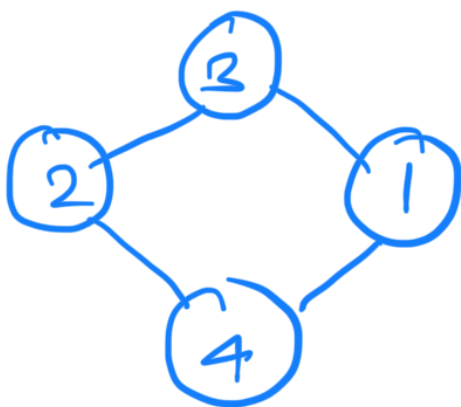
Undirected

Edges have no direction



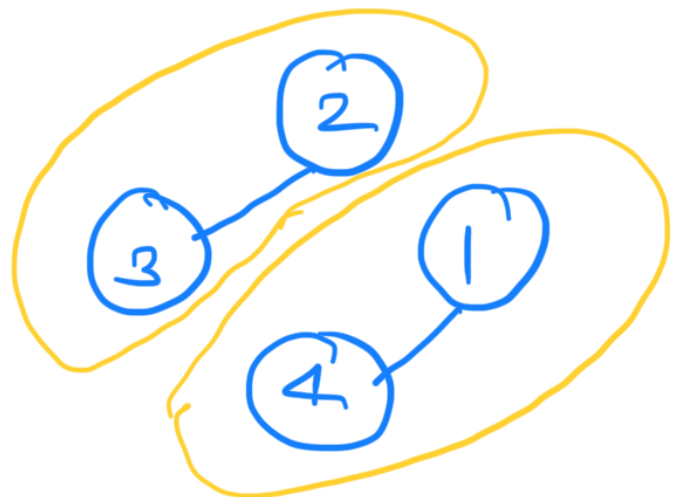
Directed

Edges have direction



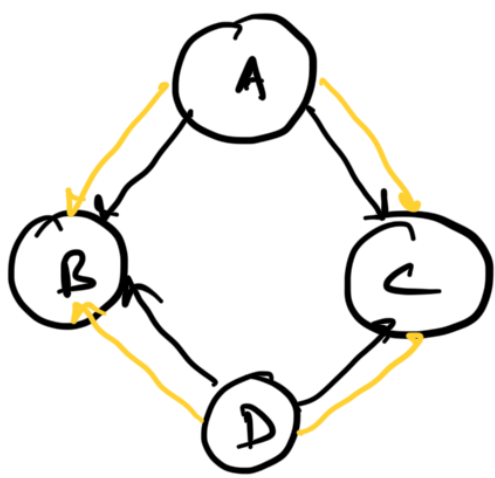
Connected graph

Every node is reachable from any vertex



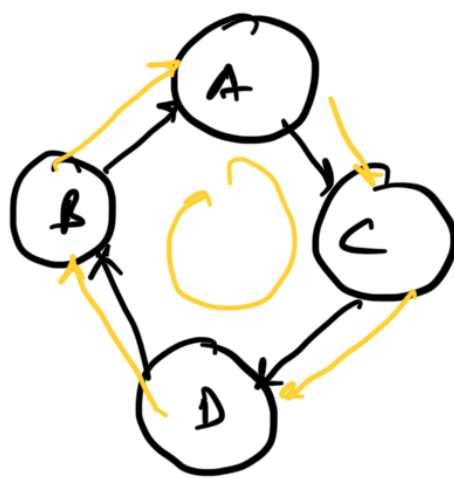
Dis Connected graph

At least one node is not reachable



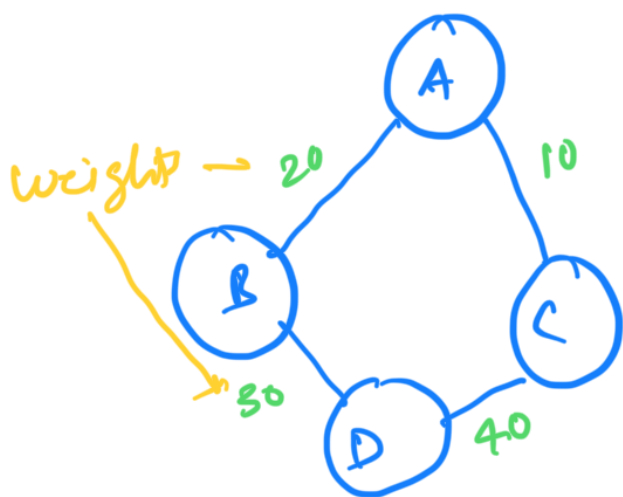
Directed Acyclic Graph (DAG)

- Directed graph with no cycle



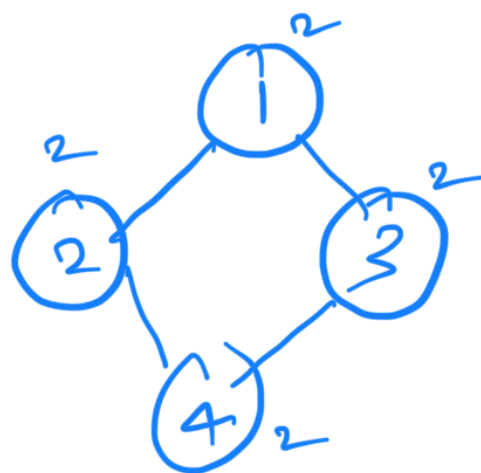
Cycle Graph

A graph where all vertices visited & form a cycle



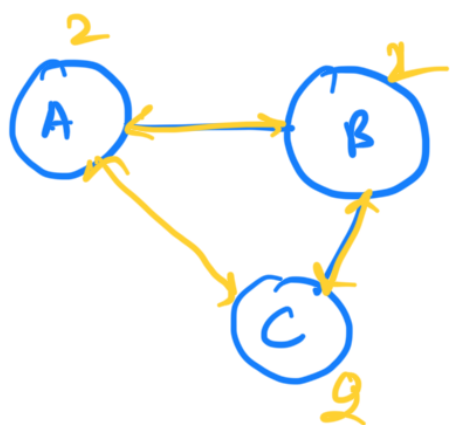
Weighted Graph

Graph where edge carry weights (weight) cost / distance

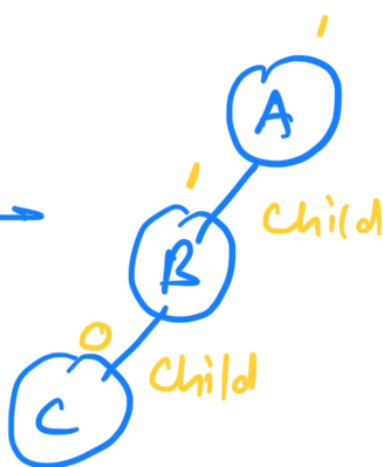


2- Regular Graph

Graph with degree of every vertex is equal to k is called k Regular Graph



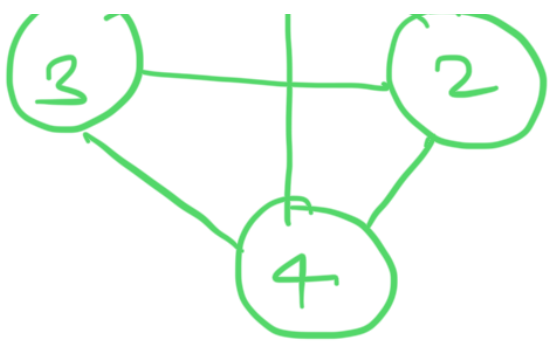
Regular Graph



degree A

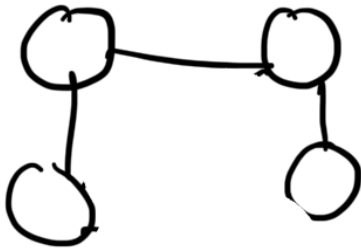


Graph in which from

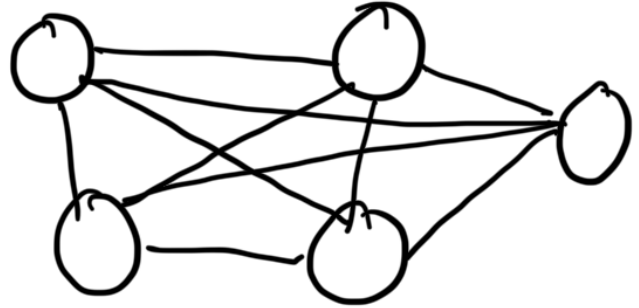


each node there is
a edge to another
node

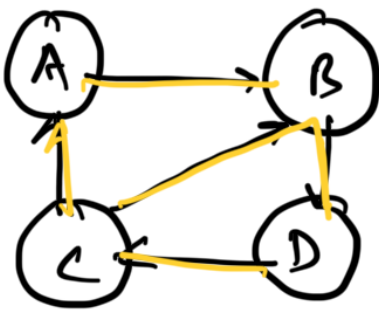
Complete Graph



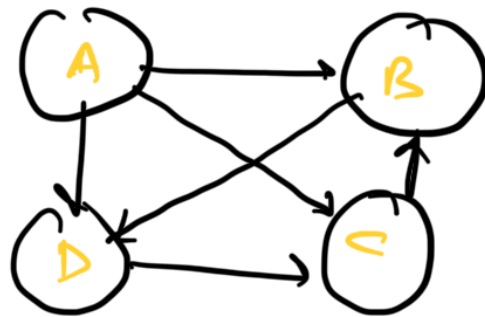
Sparse
Graph



Dense
Graph



Cyclic

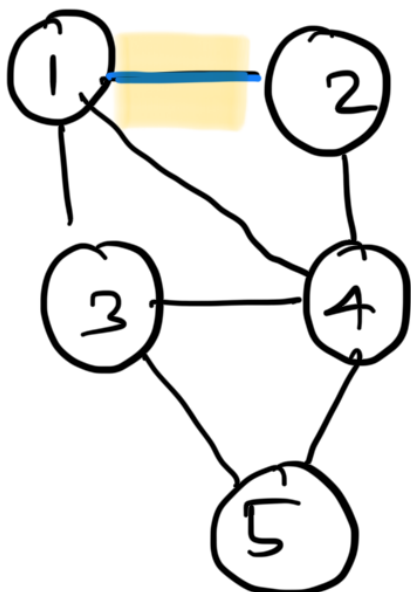


Acyclic

Representation of Graph

1. Adjacent Matrix
2. Adjacent List

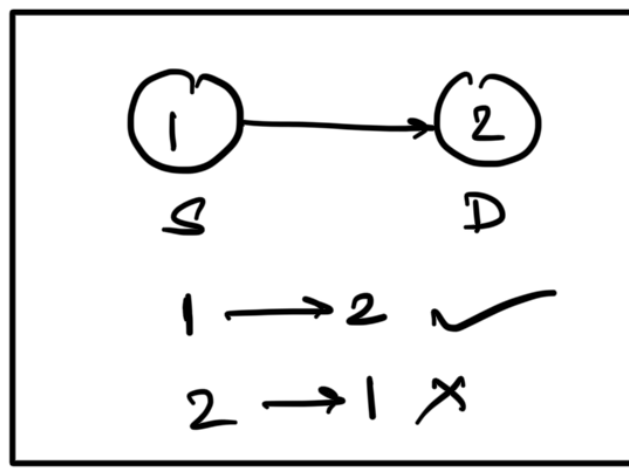
Adjacent Matrix : 2D Array



	X	1	2	3	4	5
1	0	1	1	1	0	
2	1	0	0	1	0	
3	1	0	0	1	1	
4	1	1	1	0	1	
5	0	0	1	1	0	

$$n = 5$$

Directed
graph



$$n \times n \\ 5 \times 5 \\ = n^2$$

Adjacy Matrix - Space Complexity : $O(n^2)$

→ Undirected graph, the matrix is symmetric

→ A 2D matrix where each cell in row i and col j represents the presence or absence of an edge at vertex i

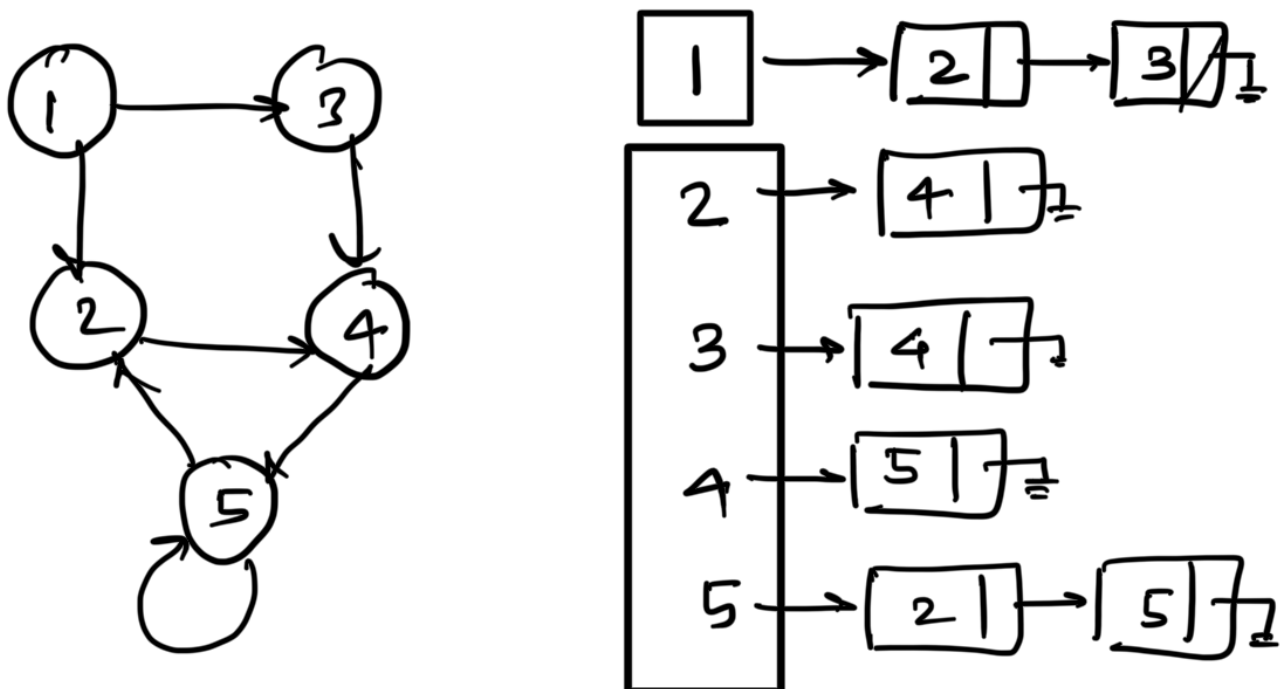
→ Time Complexity -

- Adding or checking for an edge = $O(1)$

→ Space Complexity -

- In spite of edge exist or not = $O(n^2)$

2. Adjacency List - Linked List



Space : $O(|E|)$

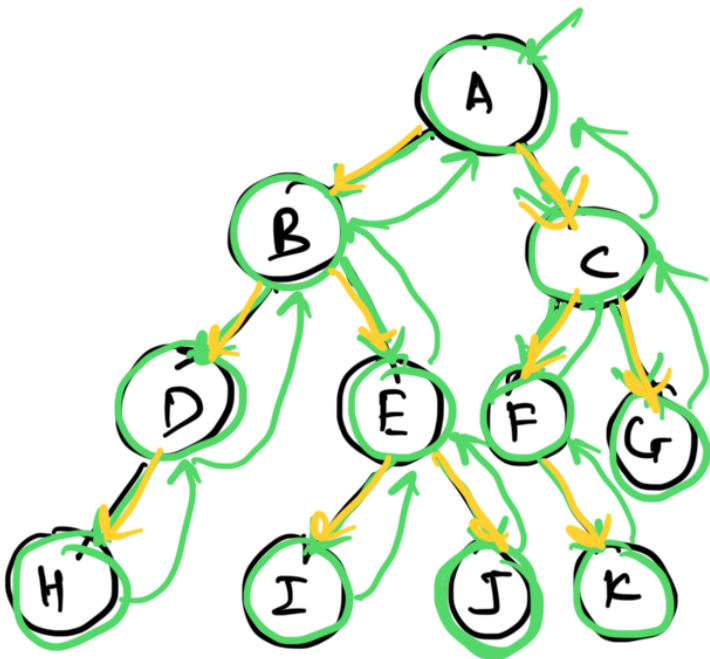
edges

- Each vertex has a list of adjacency vertex, and saving space for sparse graph.
- Time Complexity - $O(V+E)$

Adding an edge = $O(1)$

- Space Complexity - $O(|E|)$

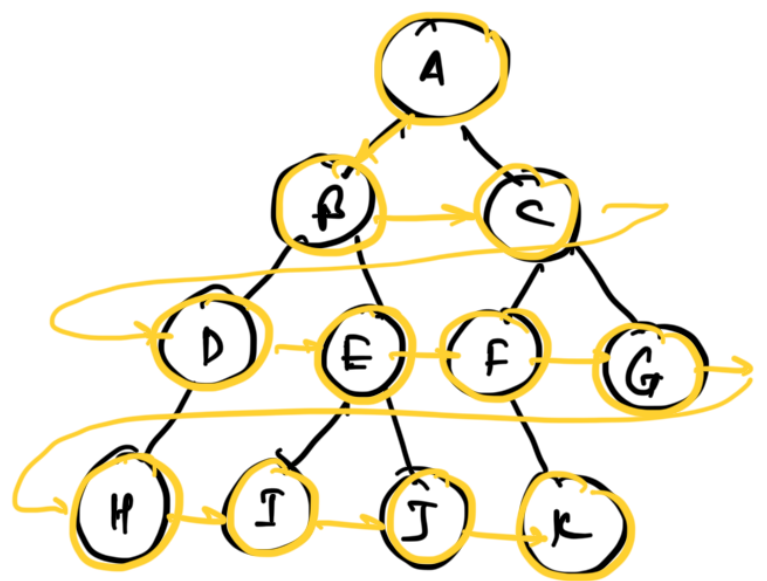
Graph Traversal Algorithms



DFS: Child (left)

A B D H E I J C F K G

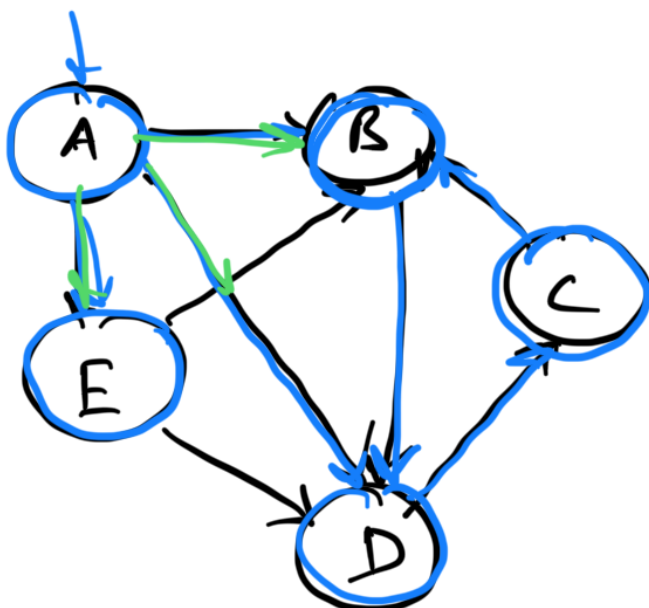
Stack:



BFS: Siblings

A B C D E F G
H I J K

Queue:



DFS: A B D C E

- ① Visit a Node
- ② Explore the Node

BFS: A B D E C

Stack

A: (B), (D), E
 B: (D)
 D: (C)
 C: (B)
 E: ~~B~~, (D)

Queue

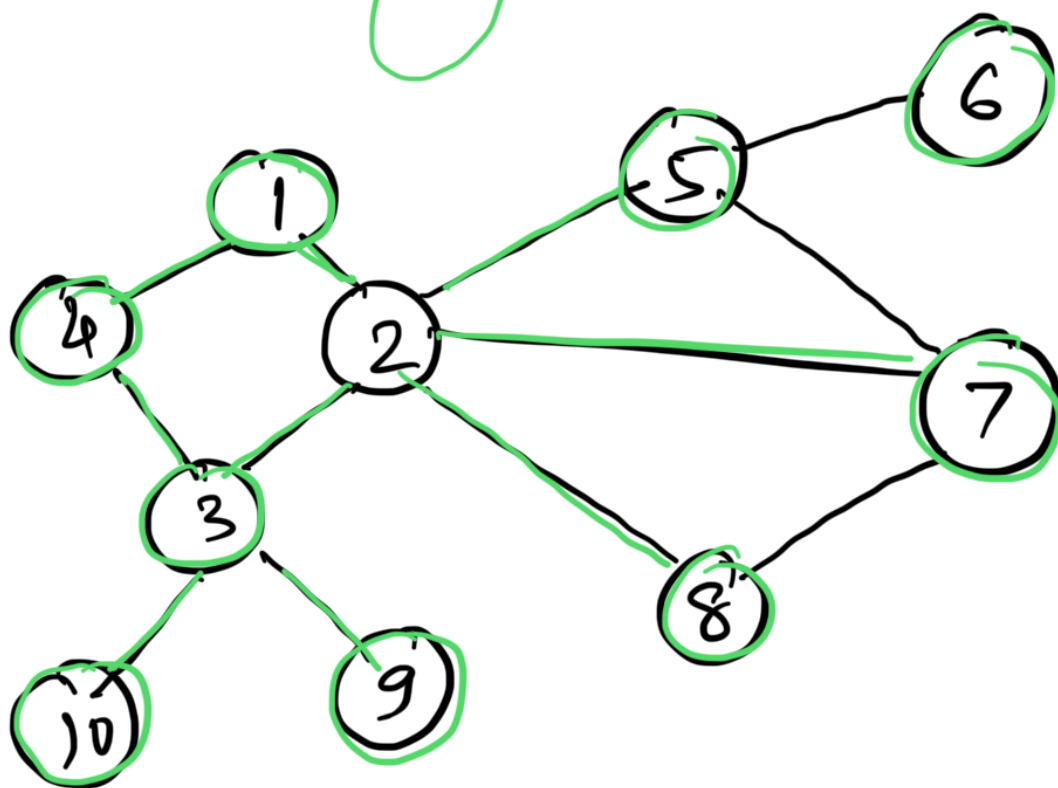
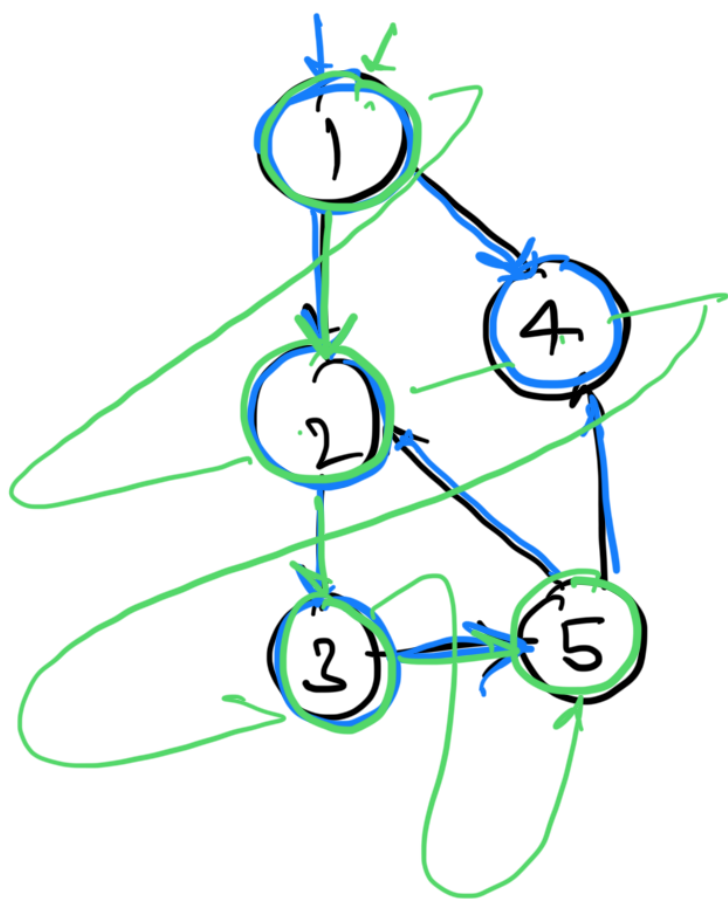
A: B, D, E
 B: ~~B~~
 D: ~~D~~
 C: ~~B~~
 E: ~~B~~, D

A: B D E
 A: D B E
 A: E B D

BFS: 1, 2, 4, 3, 5

①: 2, 4 ✓
 ②: 3 ✓
 ③: 5 ✓
 ⑤: ~~2~~, ~~4~~ ✓
 ④: —

level
 visit



1: 2, 4
 2: ~~1~~, 3, 5, 7, 8
 3: ~~2~~, ~~1~~, 9, 10
 4: ~~1~~, ~~2~~
 9: ~~3~~
 10: ~~3~~
 5: ~~2~~, 6, 7
 6: 5

DFS: 1, 2, 3, 4, 9, 10, 5, 6, 7, 8 | 7: ~~2~~, ~~3~~, 8
 BFS: 1, 2, 4, 3, 5, 7, 8, 9, 10, 6 | 8: 2, 7

BFS	DFS
 Queue	 Stack
level by level	Depth-wise exploration
Shortest Path (unweight graph)	No (No guarantee for short path)
SC: $O(V)$	SC: $O(V)$
TC: $O(V+E)$	TC: $O(V+E)$